Universidad de las Fuerzas Armadas ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software

Aplicaciones Distribuidas

Resumen del Primer Parcial

Estudiante: Luis Sagnay

Docente: Geovanny Cudco

NRC: 30731

29 de abril de 2026

Índice general

1. Conceptos **3**
1.1. Programa . . . 3
1.2. Aplicación . . . 3
1.3. Sistema . . . 3
1.4. Comparación entre los tres conceptos . . . 3
1.5. Aplicación Centralizada . . . 4
1.6. Aplicación Distribuida . . . 4

2. Evolución de los sistemas informáticos **5**
2.1. Sistemas centralizados versus distribuidos . . . 5
2.2. Cronología histórica . . . 5

3. Ventajas y desafíos de las aplicaciones distribuidas **6**
3.1. Ventajas . . . 6
3.2. Desventajas y Desafios . . . 7
3.3. Resumen comparativo . . . 7

4. Falacias de la computación distribuida **8**

5. El teorema CAP **9**
5.1. Las tres propiedades . . . 9
5.2. Bases de datos y el espectro CAP . . . 9

6. Gestión de concurrencia en aplicaciones distribuidas **10**
6.1. Conceptos fundamentales . . . 10
6.2. Hilos de ejecución en la programación distribuida . . . 10

7. Conclusiones **11**

1. Conceptos

Para hablar de aplicaciones distribuidas hay que aclarar tres términos que todo el mundo usa pero pocos distinguen con precisión, que son programa, aplicación y sistema. No son sinónimos, pero se suelen como si lo fueran.

1.1. Programa

Un programa es la unidad más básica, es un conjunto de instrucciones que la computadora ejecuta para realizar una tarea concreta. No necesita interfaz gráfica ni interacción del usuario. El comando `ls` de Linux o un script de Python que renombra archivos automáticamente son programas. Reciben una entrada, hacen algo con ella y producen una salida.

1.2. Aplicación

Programa o conjunto de programas que presupone algún mecanismo de interacción humana, ya sea una pantalla, un teclado o una voz para satisfacer una necesidad concreta. Microsoft Word, una app de mensajería o una plataforma de streaming son aplicaciones. La diferencia con el programa no está en el tamaño del código sino en para quién está pensado, que es el usuario final. Necesitan un hardware y sistema operativo como base.

1.3. Sistema

Un sistema es algo más grande que la suma de sus partes. Combina hardware, software, middleware, redes y personas para lograr objetivos que ninguno de sus componentes podría alcanzar solo. Un sistema de gestión hospitalaria, por ejemplo, integra aplicaciones para médicos, pacientes y administrativos, una base de datos centralizada, servidores, red y personal. Cada pieza funciona de forma independiente, pero el sistema solo tiene sentido cuando todo trabaja junto.

1.4. Comparación entre los tres conceptos

La tabla 1.1 resume las diferencias más importantes.

Cuadro 1.1: Comparación entre programa, aplicación y sistema

Aspecto	**Programa**	**Aplicación**	**Sistema**
Interacción con usuario	No necesaria	Siempre presente	Variable
Interfaz gráfica	Opcional	Habitual	Depende
Alcance	Tarea concreta	Necesidad del usuario	Objetivo organizacional
Ejemplo	Script de Python	Microsoft Word	Sistema hospitalario
Complejidad	Baja	Media	Alta

.

1.5. Aplicación Centralizada

Es aquella donde toda la lógica de negocio, el almacenamiento de datos y el control se gestionan desde un **único servidor central**. Los clientes actúan como terminales que acceden a este núcleo; si el servidor falla, el servicio se interrumpe por completo para todos los usuarios.

- **Ejemplo:** Un Sistema de Gestión Académica Local (SAE) donde toda la información escolar reside en un solo servidor físico dentro de la institución.
- **Ejemplo:** Puntos de venta (POS) locales que dependen de un servidor único en la tienda para facturar.

1.6. Aplicación Distribuida

Son sistemas cuyos componentes de software se ejecutan en **múltiples máquinas (nodos)** interconectadas por una red. Estos componentes cooperan entre sí en entornos y plataformas separadas para ofrecer una funcionalidad unificada al usuario final. A diferencia del modelo centralizado, la carga y la responsabilidad se reparten entre los distintos nodos del sistema.

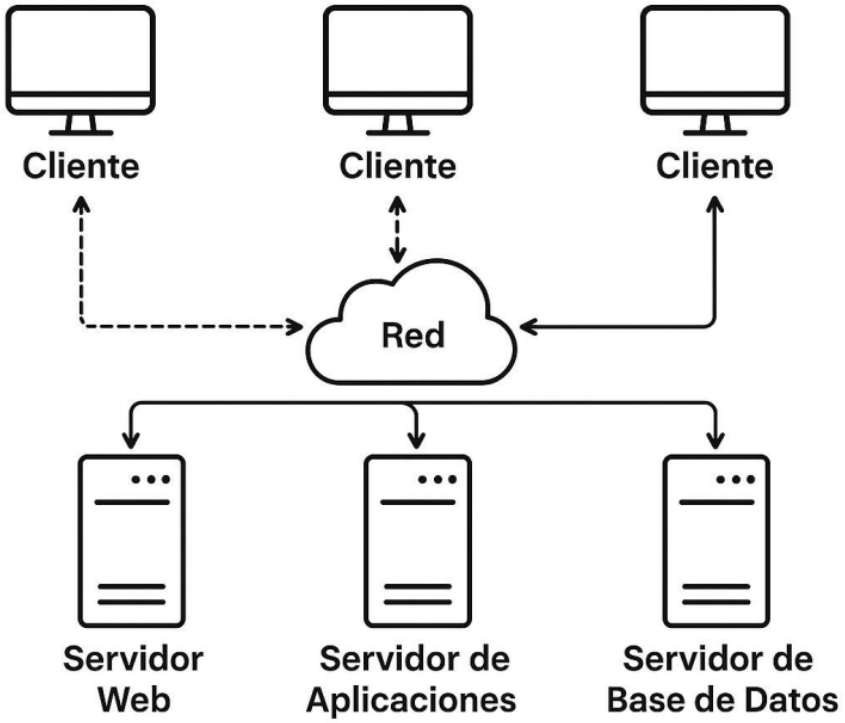

Figura 1.1: Aplicación Distribuida

2. Evolución de los sistemas informáticos

2.1. Sistemas centralizados versus distribuidos

En un sistema centralizado todo el procesamiento ocurre en un único servidor. Los usuarios acceden mediante terminales que solo muestran resultados. Este modelo dominó en los años 60 y 70, simple de administrar, pero con un punto único de fallo y límites físicos de crecimiento.

Los sistemas distribuidos reparten el procesamiento entre múltiples máquinas conectadas en red. El usuario ve una sola aplicación, pero detrás hay varios servidores colaborando. El reto es que esa colaboración parezca invisible.

2.2. Cronología histórica

Cuadro 2.1: Hitos en la evolución de los sistemas distribuidos

Año	**Evento**	**Impacto**
1961	CTSS (MIT)	Primer sistema operativo multiusuario y multitarea
1969	ARPANET / Unix	Precursor de Internet; base para sistemas operativos modernos
1972	Correo electrónico	Primera aplicación distribuida de uso generalizado
1980	Ethernet / Cliente-Servidor	Redes locales corporativas; modelo cliente-servidor estándar
1982	PODC (conferencia)	Sistemas distribuidos como disciplina académica formal
1989	CORBA (especificación)	Estándar para comunicación entre objetos distribuidos
1993	CGI	Páginas web dinámicas; base de la web moderna
1995	Java RMI	Invocación de métodos remotos entre JVMs
2000	REST (Roy Fielding)	Estilo arquitectónico para servicios web sobre HTTP
2001–03	SOAP	Estándar para servicios web basados en XML
2005	SOA	Sistemas como colecciones de servicios reutilizables
2007–08	Cloud computing	Google App Engine; plataforma como servicio
2013	Docker	Contenedores; despliegue consistente entre entornos
2014	AWS Lambda	Computación serverless; infraestructura gestionada por el proveedor
2015	Kubernetes	Orquestación de contenedores a escala
2020s	Cloud-Native / GitOps	Aplicaciones diseñadas para cloud; infraestructura gestionada en Git

3. Ventajas y desafíos de las aplicaciones distribuidas

Las aplicaciones distribuidas tienen ventajas reales, pero también costos reales.

3.1. Ventajas

Cuadro 3.1: Ventajas de los sistemas distribuidos

Ventaja	Descripción
Escalabilidad horizontal	La escalabilidad horizontal agrega nodos al clúster. Es especialmente útil en escenarios con carga fluctuante, por ejemplo, durante el Black Friday el tráfico puede crecer órdenes de magnitud respecto a un día normal, y la arquitectura lo absorbe ajustando recursos dinámicamente. Cuando pasa el pico, se reducen. Solo se paga por lo que se usa.
Alta disponibilidad y tolerancia a fallos	La tolerancia a fallos se logra mediante redundancia, donde varios nodos realizan las mismas funciones, de modo que si uno cae los demás asumen su carga de forma automática. El patrón Circuit Breaker aplica este principio, cuando detecta fallos repetidos en un servicio, corta temporalmente la comunicación para no saturarlo. En la banca, por ejemplo, si un servidor de base de datos falla, el servidor redundante toma el relevo.
Procesamiento paralelo	Dividir una tarea entre múltiples nodos reduce el tiempo total de ejecución de forma drástica. Esto es indispensable para Big Data e inteligencia artificial, donde los volúmenes de datos hacen inviable el procesamiento secuencial. Apache Spark distribuye terabytes entre clústeres de cientos de máquinas y completa en minutos lo que en un solo servidor llevaría días.
Compartición de recursos	Una grid de cómputo permite que varias organizaciones compartan equipamiento especializado (GPUs de alto rendimiento, almacenamiento masivo) que ninguna podría costear de forma individual. En entornos académicos y de investigación el costo del equipamiento especializado frecuentemente supera los recursos disponibles de cada institución por separado, pero se vuelve accesible de forma compartida.
Flexibilidad tecnológica	Cada servicio usa la tecnología más adecuada para su función, por ejemplo Python para procesamiento de imágenes, Java para transacciones. Se puede adoptar tecnología nueva de forma incremental sin reescribir el sistema completo.

3.2. Desventajas y Desafios

Cuadro 3.2: Desventajas de los sistemas distribuidos

Desventaja	Descripción
Complejidad	La comunicación entre servicios es asíncrona, puede fallar, llegar tarde o duplicarse. Un error puede aparecer en un servicio cuya causa está en otro completamente distinto. Depurar eso no es trivial.
Costos operativos	Más nodos implica más tráfico de red, más herramientas de monitoreo, más medidas de seguridad y personal especializado. Los costos operativos frecuentemente superan los de desarrollo inicial.
Sobrecarga de red	Cada comunicación requiere serializar datos, transmitirlos y cifrarlos. Si los protocolos no se diseñan bien, esa sobrecarga puede comerse los beneficios del procesamiento distribuido.
Superficie de ataque	Cada nodo adicional es un punto potencial de vulnerabilidad. La seguridad tiene que pensarse en capas, protegiendo cada componente y cada canal de comunicación.
Consistencia de datos	Cuando hay una partición de red hay que elegir entre consistencia y disponibilidad (teorema CAP, cap. 5). La mayoría opta por consistencia eventual, lo que puede no ser aceptable en todos los contextos.

3.3. Resumen comparativo

Cuadro 3.3: Ventajas y desventajas de los sistemas distribuidos

Ventajas	**Desventajas**
Escalabilidad horizontal	Mayor complejidad de desarrollo
Alta disponibilidad y tolerancia a fallos	Costos operativos elevados
Procesamiento paralelo	Sobrecarga de red y latencia
Compartición eficiente de recursos	Superficie de ataque más grande
Flexibilidad tecnológica por servicio	Dificultad para mantener consistencia fuerte

4. Falacias de la computación distribuida

En 1990, Peter Deutsch y otros investigadores de Sun Microsystems describieron lo que llamaron las ocho falacias de la computación distribuida y hasta ahora siguen siendo relevantes, quizás más que nunca.

Cuadro 4.1: Las ocho falacias de la computación distribuida

Falacia	**Por qué es un problema**
La red es confiable	Las redes fallan. Cortes, hardware defectuoso, proveedores con problemas. Hay que diseñar para el fallo, no para la excepción.
La latencia es cero	La comunicación entre nodos siempre tarda. En aplicaciones financieras de alta frecuencia, microsegundos importan.
El ancho de banda es infinito	Cuando varios servicios compiten por red, los cuellos de botella aparecen donde no se esperaban.
La red es segura	El tráfico puede interceptarse (sniffing), modificarse (Man in the Middle) o bloquearse (DDoS). Cifrado y autenticación no son opcionales.
La topología no cambia	Las redes cambian por fallos, actualizaciones y reorganizaciones. Los servicios necesitan descubrimiento dinámico, no direcciones fijas.
Hay un solo administrador	En entornos con múltiples organizaciones o proveedores, el control nunca es único. Las políticas de seguridad tienen que contemplar eso.
El costo de transporte es cero	Transferir datos tiene costos reales: ancho de banda, latencia acumulada, facturación de cloud.
La red es homogénea	Los entornos reales mezclan dispositivos de distintos fabricantes, capacidades y versiones. Las abstracciones tienen que manejar esa diversidad.

Lo interesante de estas falacias no es que sean sorprendentes, sino que siguen apareciendo en proyectos nuevos.

Edge Computing y las falacias

El *edge computing* acerca el procesamiento a la fuente de los datos: dispositivos IoT, vehículos autónomos, equipamiento industrial. Esto reduce la dependencia de nubes centralizadas y mitiga algunas de las falacias, especialmente la de latencia y ancho de banda. A cambio, introduce complejidad nueva al dividir el procesamiento entre el borde y la nube, y hace que la gestión de la topología sea todavía más dinámica.

5. El teorema CAP

El teorema CAP, propuesto por Eric Brewer en el año 2000, establece un límite matemático para lo que un sistema distribuido puede garantizar. Más de veinte años después sigue siendo el marco de referencia para decidir qué base de datos usar o cómo diseñar la arquitectura de un sistema.

5.1. Las tres propiedades

Cuadro 5.1: Las tres propiedades del teorema CAP

Propiedad	**Qué garantiza**	**Cuándo es crítica**
Consistencia (C)	Todos los nodos ven el mismo dato al mismo tiempo	Sistemas financieros, inventarios
Disponibilidad (A)	El sistema responde a todas las peticiones aunque algunos nodos hayan fallado	Redes sociales, comercio electrónico
Tolerancia a particiones (P)	El sistema sigue operando aunque se pierda comunicación entre nodos	Siempre necesaria en sistemas distribuidos reales

5.2. Bases de datos y el espectro CAP

Cuadro 5.2: Clasificación de bases de datos según el teorema CAP

Tipo	**Ejemplos**	**Características**
CA	MySQL, PostgreSQL, Oracle	Transacciones ACID, operación monolítica o con replicación síncrona. Sin tolerancia a particiones de red.
CP	Redis, MongoDB, HBase, Zookeeper	Priorizan consistencia. Durante una partición pueden quedar inaccesibles, pero los datos son siempre correctos.
AP	Cassandra, DynamoDB	Priorizan disponibilidad. Aceptan consistencia eventual: los datos se sincronizan con el tiempo. Adecuadas cuando la disponibilidad supera a la exactitud inmediata.

La elección depende del contexto. Un banco no puede permitirse mostrar saldos incorrectos, así que elige CP. Una red social puede mostrar un contador de me gusta con un segundo de retraso sin que nadie lo note, así que elige AP.

6. Gestión de concurrencia en aplicaciones distribuidas

La concurrencia es la capacidad de un sistema para ejecutar múltiples tareas al mismo tiempo, ya sea en paralelo real o mediante intercalación. En sistemas distribuidos esto ocurre a dos niveles: dentro de cada nodo y entre los distintos nodos del sistema.

6.1. Conceptos fundamentales

La concurrencia mejora el rendimiento al aprovechar múltiples núcleos de CPU o conexiones de red simultáneas. Pero introduce tres problemas que hay que gestionar explícitamente:

Condición de carrera: ocurre cuando dos tareas acceden al mismo recurso compartido y el resultado depende de cuál llegue primero. Puede producir estados inconsistentes en bases de datos o transacciones financieras incorrectas. Lo complicado es que estos errores son difíciles de reproducir, porque dependen del timing exacto.

Deadlock: dos o más procesos se bloquean mutuamente, cada uno esperando que el otro libere un recurso. En sistemas distribuidos el problema se escala porque pueden involucrar servicios en distintos nodos.

Inanición: un proceso nunca obtiene acceso a los recursos que necesita porque otros tienen siempre prioridad. No produce un error visible de inmediato, pero degrada el rendimiento de forma silenciosa.

6.2. Hilos de ejecución en la programación distribuida

Los hilos son unidades básicas de concurrencia; en sistemas distribuidos pueden pausarse al esperar respuestas remotas, lo que introduce latencia y overhead de red.

En lenguajes como Java o Go se crean mediante `Thread` o `goroutines`, y suelen gestionarse en pools para mejorar la escalabilidad. Herramientas como Kubernetes orquestan contenedores que ejecutan aplicaciones apoyadas en pools de hilos distribuidos.

7. Conclusiones

Trabajar con sistemas distribuidos es aprender a gestionar el caos. No es que sean una opción mejor o peor que los centralizados; es que son la única forma de que una aplicación soporte el tráfico y las exigencias de muchas aplicaciones de la actualidad. Pero hay que tomar en cuenta que la complejidad que se ahorra en el código suele aparecer en la infraestructura.

Con esta información queda claro que conceptos como el teorema CAP o las falacias de la red son el motivo real por el que un sistema se cae cuando menos te lo esperas. Al final, diseñar estas aplicaciones no se trata de encontrar la herramienta perfecta, sino de entender qué sacrificios estás dispuesto a hacer (consistencia frente a disponibilidad, por ejemplo) para que el sistema funcione.

Es interesante ver cómo herramientas como Docker o Kubernetes, que nacieron para facilitarnos la vida, han terminado creando sus propios problemas. La complejidad no desaparece, solo se mueve de lugar. Por ello en la actualidad no se debe de enfocarse en saber usar la última tecnología de moda, sino en comprender qué está pasando realmente debajo de todo.